



Infrastructure Provisioning for SpeechLab Applications with Terraform

Chan Ming Han

U2120477J

Supervisor: Prof Chng Eng Siong

A Final Year Report submitted to School of Computer Science and
Engineering, Nanyang Technological University in partial fulfilment of the
requirements for the Degree of

**BACHELOR OF ENGINEERING SCIENCE (COMPUTER
ENGINEERING)**

2024/2025 Semester 2

Abstract

As companies move toward containerisation technologies to improve scalability and portability in their applications, Kubernetes has emerged as the industry standard for orchestrating containerised workloads. Despite its advantages, deploying containerised workloads to the cloud is highly complex, and resource intensive. Developers and engineer also need to master the various cloud technologies, such as security and networking requirements, often diverting focus from core application development. This project introduces cloudkube, a lightweight, Python-based command-line tool designed to automate the deployment of Kubernetes clusters in the cloud. Inspired by minikube and leveraging Infrastructure as Code (IaC) principles with Terraform, cloudkube abstracts the complexities of cloud provisioning and Kubernetes setup. It enables developers to focus on core application development, rather than cloud or deployment configurations.. The tool supports customizable configurations, automated local environment setup, and idempotent resource provisioning while ensuring low-cost prototyping by leveraging AWS Free Tier resources. Through cloudkube, deployment times are significantly reduced, and infrastructure provisioning is made accessible to users without deep expertise in cloud technologies. This paper details the tool's architecture, implementation, and workflow, showcasing its potential to enhance productivity, reduce operational overhead, and accelerate the deployment of containerised applications to the cloud.

Keywords: Containerization, Cloud Automation, Infrastructure as Code (IaC), Terraform, Cloud Deployment, Kubernetes Orchestration, Developer Productivity, Lightweight CLI Tools, Cloud-Native Applications

Acknowledgement

I would like to express my deepest gratitude to my academic advisors and mentors, including but not limited to Prof Chng Eng Siong, Research Assistant Vu Thy Ly, and James. Their guidance and invaluable insights have been instrumental throughout the course of this project. Their continuous support and constructive feedback have greatly enhanced the quality of my work. I am particularly grateful for their encouragement and inspiration, which motivated me to tackle complex challenges and explore innovative solutions.

I extend my sincere thanks to my peers, who contributed through stimulating discussions and innovative ideas. Their willingness to share knowledge and exchange ideas fostered an enriching environment that enabled the successful completion of this project.

Finally, I would like to acknowledge the funding and infrastructure support provided by Nanyang Technological University. The access to the AWS cloud platform significantly facilitated the implementation and testing phases of this project. Without their generous support, this work would not have been possible.

Table of Contents

Abstract	i
Acknowledgement	ii
Lists of Figures	v
Lists of Tables	vi
1 Introduction	1
1.1 Background	1
1.2 Objective	2
1.3 Project Scope	3
1.4 Contributions	3
1.5 Project Schedule	5
1.6 Report Organisation	6
2 Literature Review	7
2.1 Infrastructure as Code (IaC)	7
2.1.1 Terraform	7
2.1.2 AWS CloudFormation	10
2.1.3 OpenTofu	10
2.1.4 Ansible	10
2.2 IaC Orchestration and Frameworks	10
2.2.1 Terragrunt	10
2.2.2 Terraspace	11

2.3	Container and Container Orchestration	11
2.3.1	Docker	11
2.3.2	Kubernetes	11
2.4	Infrastructure and Cloud	14
2.4.1	Virtual Private Cloud (VPC)	14
2.4.2	Subnets	15
2.4.3	Virtual Machines	16
2.4.4	Security Groups	16
2.4.5	Elastic Network Interfaces (ENI)	16
2.4.6	Instance Templates	16
2.4.7	Managed Kubernetes Service	17
2.4.8	File Systems	17
2.4.9	Scaling Groups	17
2.4.10	minikube	18
3	Planning and Design	19
3.1	Traditional Approaches	19
4	Implementation	20
5	Results and Analysis	21
6	Conclusion and Future Work	22
A	Appendix A: AWS CloudFormation YAML Template for EKS IPv4	R-1

Lists of Figures

1-1	Gantt Chart for Project Schedule	5
2-1	Sequence Diagram for Terraform Backend	9
2-2	Typical Kubernetes Architecture	12

List of Tables

1-1	Project Schedule	5
-----	----------------------------	---

Introduction

1.1 Background

Automatic Speech Recognition (ASR) is a transformative technology that enables machines to interpret and process human speech into text. Rooted in advancements in machine learning, signal processing, and linguistics, ASR has rapidly evolved over the past decades to become a cornerstone of human-computer interaction. At its core, ASR has allowed for machines to understand, and reproduce human speech artificially (Panda, 2018).

The use cases for ASR span a wide range of real-time applications that significantly impact various industries. In customer service, ASR powers virtual assistants and chatbots, enabling real time transcription of customer queries and automated responses. Virtual assistants like Apple's Siri, Amazon's Alexa, Microsoft's Cortana, and more also rely on ASR to process voice commands and execute tasks ranging from setting reminders to controlling smart home devices (Hoy, 2018). On our favourite video conferencing platforms such as Microsoft Teams, Zoom, or Google Meets, ASR is also integrated to provide live captions during meetings, enhancing collaboration and inclusivity (Qiao & Yijun, 2023). These examples highlight how ASR has become an integral part of our daily lives, simplifying tasks, improving accessibility, and transforming how people interact with technology.

NTU SpeechLab has also undertaken multiple projects, creating speech engines that can transcribe English, Mandarin and Singlish in real-time. These speech engines are used in various use cases including speech-to-text at call centres, live transcription during interviews, as well as medical consultations. This is done through either of two methods based on the use case. The first is batch transcription, in which a completed audio recording is analysed for transcription. The second is live transcription, in which an audio stream is analysed in real-time to provide on-the-fly transcription.

Due to the on-the-fly nature of live transcriptions, as well as the ever evolving need for ASR technologies, it is important for such applications to be built in a robust and scalable manner. These services should be highly available and scalable. Previous FYPs have also worked on containerisation and deployment of the ASR speech model via Kubernetes or Docker Swarm, as well as robust CI/CD pipelines using ArgoCD to speed up the build and deployment times the ASR speech application.

However, as the number of such applications have increased, the infrastructure requirements have also become more demanding. In the deployment of the ASR speech model, one of the greatest pain points was the amount of time and effort it took to learn cloud technologies, and ultimately deciding on the most suitable infrastructure to host the ASR application on the cloud. My mentors also shared that the DevOps portion of their work took up a sizeable amount of their time. As such, I noted the importance of a quick turnaround time for deploying a new project. Furthermore, with the industry moving toward containerisation, I noted the importance of ease

of use of container orchestration software such as Kubernetes. As such, I decided on building a terminal tool to quickly spin up an Kubernetes cluster for a quick and easy deployment.

1.2 Objective

The main objective of this project was to speed up the turnaround time for deployment of new ASR applications. The main idea of this tool is to abstract away in-depth knowledge of cloud technologies from researchers, and allow them to readily and easily deploy their ASR applications to the cloud at the press of a button. This will allow researchers to then concentrate their efforts on the development of the ML model in the ASR application, and not have to worry about low-level cloud implementation details such as creating a security group role to allow ingress/egress traffic, or configuring their local environment to hit the cluster.

Drawing inspiration from minikube, which is a quick way to spin up a local cluster, for this FYP, I have decided to build cloudkube, which quickly spins up a Kubernetes cluster in the cloud. Users then have a simple barebones Kubernetes cluster hosted in the cloud where they can do dev testing.

The functional requirements for this project are detailed as follows:

- a. cloudkube should provision infrastructure such as compute instances, storage, networking, and Kubernetes clusters in the cloud.
- b. cloudkube should be easily extendable to different cloud providers.
- c. cloudkube should create a Kubernetes environment, including deploying essential components like ingress controllers.
- d. cloudkube should allow users to easily deploy containerized ASR applications to the Kubernetes cluster.
- e. cloudkube should be able to deprovision and clean up resources without leaving anything dangling to prevent unnecessary costs.
- f. cloudkube should maintain the state of provisioned infrastructure to avoid dangling resources in the cloud.

The non-functional requirements for this project are detailed as follows:

- a. cloudkube should be easy to use, and have a gentle learning curve.
- b. cloudkube should be user-friendly, with clear command structures and help options.
- c. cloudkube, the CLI tool, should provision resources and deploy applications within a reasonable time frame.
- d. cloudkube should provision and deprovision resources idempotently and atomically.
- e. cloudkube should be able to work across different environments, including Windows, MacOS, and Linux.
- f. cloudkube should support the addition of new features or integrations.

1.3 Project Scope

The scope of this project is focused on building a minimum viable project for cloudkube, to provision infrastructure for hosting applications in a Kubernetes environment in the cloud. cloudkube aims to simplify the development process by automatic infrastructure provisioning, Kubernetes cluster setup, cloud configuration, as well as application deployment. While the potential of cloudkube is massive, with possibilities for features such as multi-cloud support, advanced monitoring integrations, extensibility of modules and an infrastructure templates, the scope of this project has been deliberately constrained due to the time frame of this FYP.

Given the limited time frame, this project focuses on delivering core functionalities, which are as follows:

- a. Efficiently spin up a Kubernetes cluster that applications can be easily deployed to.
- b. Allow some customizability of the infrastructure:
 - i. Architecture of compute instances.
 - ii. Use of ingress controller.
 - iii. Use of elastic file system for persistent volumes functionality in Kubernetes.
- c. State tracking to ensure idempotent and consistent behavior.
- d. Configuration of local environment to communicate with the Kubernetes cluster.
- e. Clean deprovisioning and tear down of resources.

1.4 Contributions

This section highlights the key contributions made during the course of this project. The key contributions to this project can be roughly broken down into two parts. The first and main part relates to the infrastructure provisioning, which allows for any containerised workload to be deployed to a Kubernetes cluster hosted on the AWS cloud. The second part relates to optimizing the deployment of the ASR application.

Contributions toward Infrastructure Provisioning Workflow

- a. **Optimized the workflow for creating Kubernetes clusters in the cloud:** In order to facilitate creating Kubernetes clusters in the cloud, I developed the cloudkube CLI tool to automate the provisioning of Kubernetes clusters, as well as their corresponding dependencies in the cloud..
- b. **Creation of a user-friendly interface:** cloudkube was created with clear command structures and help options to ensure ease of use. As a tool aimed at developers who are not experts in cloud, cloudkube was built to abstract away in-depth cloud intricacies.
- c. **Idempotency and Atomic Provisioning:** To align with the requirements stated above, cloudkube was also developed with state tracking, to ensure idempotent and atomic provisioning and deprovisioning of resources to prevent resource leaks and unnecessary costs.

-
- d. **Easy user configurability:** After speaking to developers working on SpeechLab applications, and noting the use cases of different Kubernetes workloads, cloudkube provides customization options for infrastructure components such as compute instances, ingress controllers, and persistent volumes.
 - e. **Optimization of development workflow for authenticating local machine with Kubernetes cluster:** cloudkube also facilitates the configuration of local environments to communicate and authenticate seamlessly with the Kubernetes cluster.
 - f. **Extensive testing:** Extensive testing was also conducted to ensure the reliability and performance of the cloudkube tool. This included running multiple Kubernetes workloads against the provisioned cluster, which will be explained in detail in Chapter 4.

Contributions toward Optimizing ASR Kubernetes Deployment

- a. **Optimized workflow for copying of models into compute worker instances:** The previous workflow for copying of the models into the EC2 worker instances was via secure copy scp. In this project, a more streamlined process was developed, and models were automatically pulled from cloud storage into compute instances (Song Yu, 2024).

1.5 Project Schedule

The project schedule is presented in Table 1-1. A Gantt chart for the project schedule is shown in Figure 1-1. It should be noted that these start and end dates serve as rough guidelines, especially for the implementation phase of the project.

Index	Task Name	Start Date	End Date
Preparation Phase			
1	Literature Review	31 May 2024	30 Jul 2024
2	Requirements Analysis	14 Jun 2024	15 Jul 2024
3	Requirements Refinement	15 Jul 2024	30 Jul 2024
4	System Design Development and Refinement	15 Jul 2024	30 Sept 2024
Implementation Phase			
5	Optimizing ASR Kubernetes Deployment	1 Oct 2024	7 Oct 2024
6	Provisioning Infrastructure for ASR	7 Oct 2024	9 Oct 2024
7	Development of main CLI interface of ccloudkube	1 Oct 2024	31 Oct 2024
8	Ingress Controller Implementation	1 Nov 2024	15 Nov 2024
9	Persistent Volumes Implementation	15 Nov 2024	29 Nov 2024
10	ccloudkube interactive setup wizard	15 Nov 2024	29 Nov 2024
11	ccloudkube commands setup	2 Dec 2024	16 Dec 2024
Testing Phase			
12	ASR Kubernetes Workload	16 Dec 2024	15 Jan 2025
13	Multi Service Kubernetes Workload	16 Dec 2024	15 Jan 2025
Documentation Phase			
14	Project Proposal	12 Aug 2024	2 Sept 2024
15	Interim Report	16 Dec 2024	30 Jan 2025
16	Final Report	30 Jan 2025	24 Mar 2025
17	Presentation	24 Mar 2025	14 May 2025

Table 1-1: Project Schedule

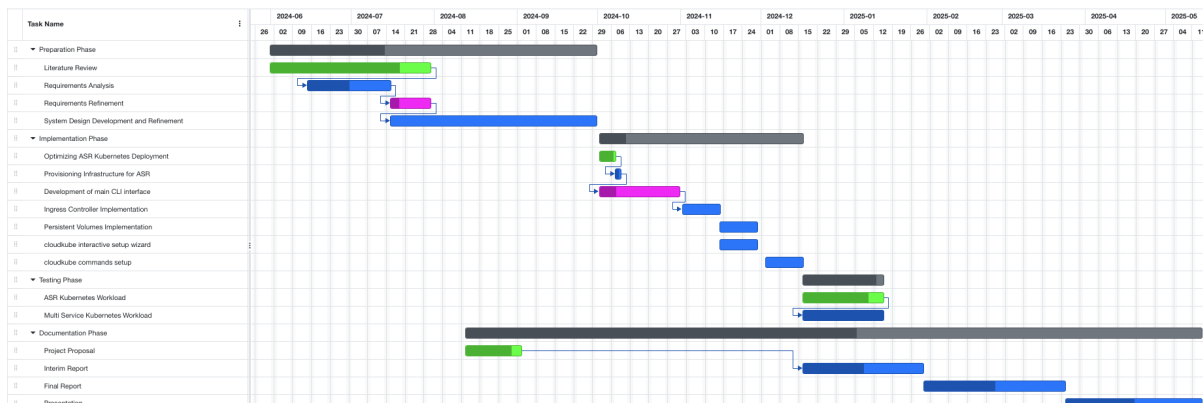


Figure 1-1: Gantt Chart for Project Schedule

1.6 Report Organisation

This report is structured into 5 different chapters, each detailing different aspects of the project.

Chapter 1 - Introduction: This chapter presents the background and introduction of the project. In this chapter, I elucidate some of the motivations for this project, and a brief overview on the intended outcome of this project.

Chapter 2 - Literature Review: This chapter reviews relevant technologies, and builds a foundational understanding for technical terminology used in future chapters. For completeness, this chapter will also discuss some of the pros and cons of each technology, which become relevant in later chapters.

Chapter 3 - Planning and Design: This chapter focuses on the analysis and design of the proposed solutions. In this chapter, I explore the current methods for handling containerised workloads, and show an optimized workflow for my proposed application.

Chapter 4 - Implementation: This chapter delves into the implementation of the proposed solutions. In this chapter, I detail the code and architecture used for `cloudkube`.

Chapter 5 - Results and Analysis: This chapter discusses the results of my proposed application, `cloudkube`. In this chapter, we will see the deployment of two applications to a Kubernetes environment provisioned by `cloudkube` - 1 ASR Application, and 2 Multi-service Kubernetes Application.

Chapter 6 - Conclusion and Future Work: This chapter presents the conclusions and recommendations for future work.

Literature Review

To determine the most optimal design and implementation tool, I conducted a comprehensive evaluation of the various cloud technologies. This involved exploring the capabilities of each cloud technology, and diving into the supporting documentation. By understanding the objectives, strengths, and weaknesses of each of the different technologies, I was able to narrow down on the problem that I was trying to solve, and select the best tools that will align with the requirements of the project.

2.1 Infrastructure as Code (IaC)

According to AWS, “Infrastructure as Code (IaC) is the ability to provision and support your computing infrastructure using code instead of manual processes and settings.”. In the typical software development life cycle, when it gets to deploying applications, developers have to regularly set up, update, and maintain infrastructure in order to develop, test, and deploy applications [3].

With more and more applications coming up, and companies focusing on high velocity, it has become increasingly important for these manual tasks like infrastructure provisioning to be automated. Furthermore, manual infrastructure provisioning is also more error prone, especially when managing applications at scale (AWS, n.d.). As such, more and more IaC tools have come up, so that infrastructure can be provisioned reliably, quickly, and unambiguously to support our applications in the cloud. Next, we will discuss some of the common IaC tools, their uses cases, strengths as well as pitfalls.

2.1.1 Terraform

Perhaps one of the most popular IaC tools is Terraform. Terraform is an open-source IaC tool created by HashiCorp, which allows users to define and provision infrastructure declaratively. With Terraform, one can specify infrastructure resource in human-readable configuration files that can be reused, distributed, and modified [5].

Benefits

1. **Flexibility and Scalability:** Terraform integrates with over 1700 service providers governing different types of resources and services [5], which means that developers can automate the deployment and management of a wide variety of resources ranging from compute instances to storage solutions.
2. **Cloud Agnostic:** Terraform is cloud agnostic, meaning that a single configuration can manage multiple providers and handle cross-cloud dependencies.

-
3. **Open Source:** Terraform is open source and free to use.
 4. **State Management:** Terraform manages its own state, meaning that cloud resources created and destroyed are all kept track of in a Terraform state file. This eliminates the worry of dangling or failed-to-provision resources.

How Terraform Works

1. **Providers:** At a high level, Terraform creates resources based on the targets exposed application programming interfaces (APIs). Communication with these APIs happens through providers, which are plugins that interact with various platforms and manage their resources. These providers can be seen as a logical abstraction of an upstream API. Today, Terraform supports over 1800 providers, including major cloud providers such as AWS and GCP, as well as providers for Docker, Kubernetes, and more [6]. Providers are normally defined in a top-level `terraform.tf` file as shown below:

```
terraform {  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.47.0"  
    }  
  }  
  required_version = "~> 1.3"  
}
```

2. **terraform plan:** This command creates an execution plan, which generates a preview of the changes that Terraform plans to make to your infrastructure [9]. By default, the following steps are taken:
 - (a) Reads the current state of any already-existing remote objects to ensure that the Terraform state is up-to-date.
 - (b) Compares the current configuration to the prior state and generates a difference between the current and prior state.
 - (c) Proposes a set of changes that, if applied, will make the remote objects match the configuration.
3. **terraform apply:** This command executes the actions proposed in a Terraform plan [9]. By default, the following steps are taken:
 - (a) All the steps as detailed in the above section on `terraform plan`.
 - (b) Prompt for plan approval. For auto-approval, the flag `--auto-approve` can also be passed to the `terraform apply` command.
 - (c) Undertakes the actions as indicated in the plan to modify existing cloud infrastructure to meet the desired current state.
 - (d) Modifies the state file to match the current state.

4. **terraform destroy:** This command is used as a way to destroy all remote objects managed by a particular Terraform configuration [9].
5. **The Terraform State File:** To make all of the above possible, Terraform needs to maintain some kind of state, so that it is aware of the current state of resources on the cloud. This is done through the `terraform.tfstate` file, which is automatically generated/modified on every Terraform command that mutates state. For local development of Terraform, the `terraform.tfstate` file is generated locally. For production and other higher environments, it is standard practice for the `terraform.tfstate` file to be stored in the cloud, such as an S3 bucket, with DynamoDB for locking, to ensure effective version control [7].

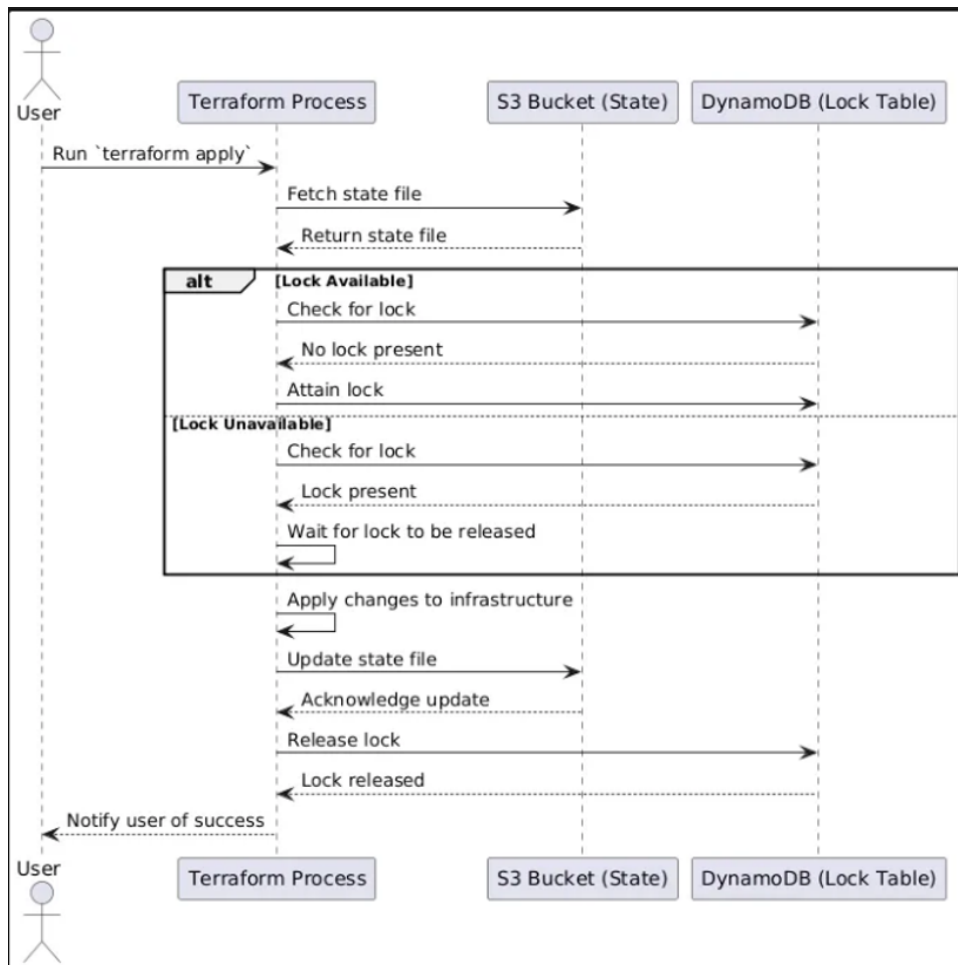


Figure 2-1: Sequence Diagram for Terraform Backend

The above diagram shows the typical workflow in handling terraform state files. In summary, state files are usually saved to some storage such as an S3 bucket. Access to these state files are then synchronized via a lock, which is commonly implemented in DynamoDB [7]. This allows race-free interaction with the provisioned infrastructure.

2.1.2 AWS CloudFormation

AWS CloudFormation is an IaC tools that allows one to manage infrastructure and automate any deployments using code [2]. CloudFormation is also done declaratively, and written in familiar languages such as YAML or JSON. This makes managing, scaling, and automating AWS resources fast and straightforward. However, the greatest downside is that CloudFormation is part of the AWS stack, and is not cloud agnostic. This means that developers are reliant on the AWS cloud should they choose to use AWS CloudFormation.

2.1.3 OpenTofu

OpenTofu is yet another open-source IaC tool designed to help with the provisioning and management of infrastructure [8]. It holds many similarities to Terraform as discussed above, but has an added feature of allowing for state encryption. There are also some differences in their licensing which will not be discussed here.

2.1.4 Ansible

Ansible is an open-source automation tool used for configuration management, application deployment, and task automation. Unlike Terraform and CloudFormation, which are declarative, Ansible uses an imperative approach to define the desired state of the infrastructure. The greatest upside of Ansible is that it is agentless, meaning it does not require any software to be installed on the target machines.

2.2 IaC Orchestration and Frameworks

IaC orchestration is used to enforce good programming practices, improving code readability and quality. This provides additional structure, automation, and improves maintainability for complex infrastructure. These tools provide a significant advantage over pure Terraform by improving code readability, promoting consistency across projects, and enabling faster collaboration and development in multi-environment setups.

Several IaC orchestration tools have also been built on top of terraform, such as terragrunt and terraspace. While an IaC orchestration tool will not be used in the implementation due to the main aim of keeping cloudkube portable and easily distributable, it is important to note how these tools fit into the whole cloud ecosystem.

2.2.1 Terragrunt

Terragrunt is a thin wrapper that is written on top of terraform, that provides some tooling and opinions [12]. Terragrunt also provides a way to keep your code DRY by using the generate helper method. Terragrunt also helps in the automated creation of backends, which can be a

chore if using raw terraform. Lastly, terragrunt allows for the injection of CLI hooks, allowing the utility to execute custom actions before or after terraform commands

2.2.2 Terraspace

Terraspace is a framework, which allows developer to dynamically generate terraform project in a centralized manner [13]. It is highly opinionated, and strongly enforces modularity and DRYness through stacks. On top of all the benefits offered by terragrunt, terraspace also allows the easy segregation of development and high-level environments through terraspace layering, which allows for the usage of the same code to deploy and create multiple environments.

2.3 Container and Container Orchestration

Containerisation has become the standard practice of the industry today. Allowing for quicker software development cycles, together with more efficient CI/CD practices and portability, containerisation provides us a powerful, and agile way to manage our applications [14]. Along with the growing complexity of distributed systems, container orchestration software such as Kubernetes are also used to automate deployment, handle scalability and fault tolerance, as well as efficient resource utilization. This chapter will discuss some of the common container and container orchestration software, which are pertinent to this project given that most SpeechLab applications have been containerised for the cloud

2.3.1 Docker

Docker is a containerisation software which aids in development, shipping, and deployment. Docker enables the separation of application from infrastructure so that software can be delivered quickly and reliably [1]. With docker, code, as well as version-locked dependencies are baked into a container image. This means that anyone building from this image on any machine in the world, regardless of architecture, will build the same thing, and run the same application. This creates reproducible environments of an application based on a single Docker Image.

2.3.2 Kubernetes

Kubernetes is an open-source container orchestration software that helps to automate deployment, scale, and manage containerised applications. Kubernetes provides a framework to run distributed systems resiliently, taking care of scaling and failover of the application [10].

Key Features of Kuberentes

1. **Self-healing:** Kubernetes restarts, replaces, or kills containers that are unhealthy, and doesn't advertise these containers to clients until they are ready to serve.

2. **Storage Orchestration:** Kubernetes allows the automatic mounting of a storage system of the developers choice.
3. **Horizontal Scaling:** Kubernetes can scale up or down an application with a simple command, UI, or automatically based on user-defined parameters.
4. **Service Discovery and Load Balancing:** Kubernetes can expose a container using the DNS name of the container, or using its own IP address. Kubernetes can also load balance and distribute network traffic to ensure deployment stability.

Kubernetes Architecture

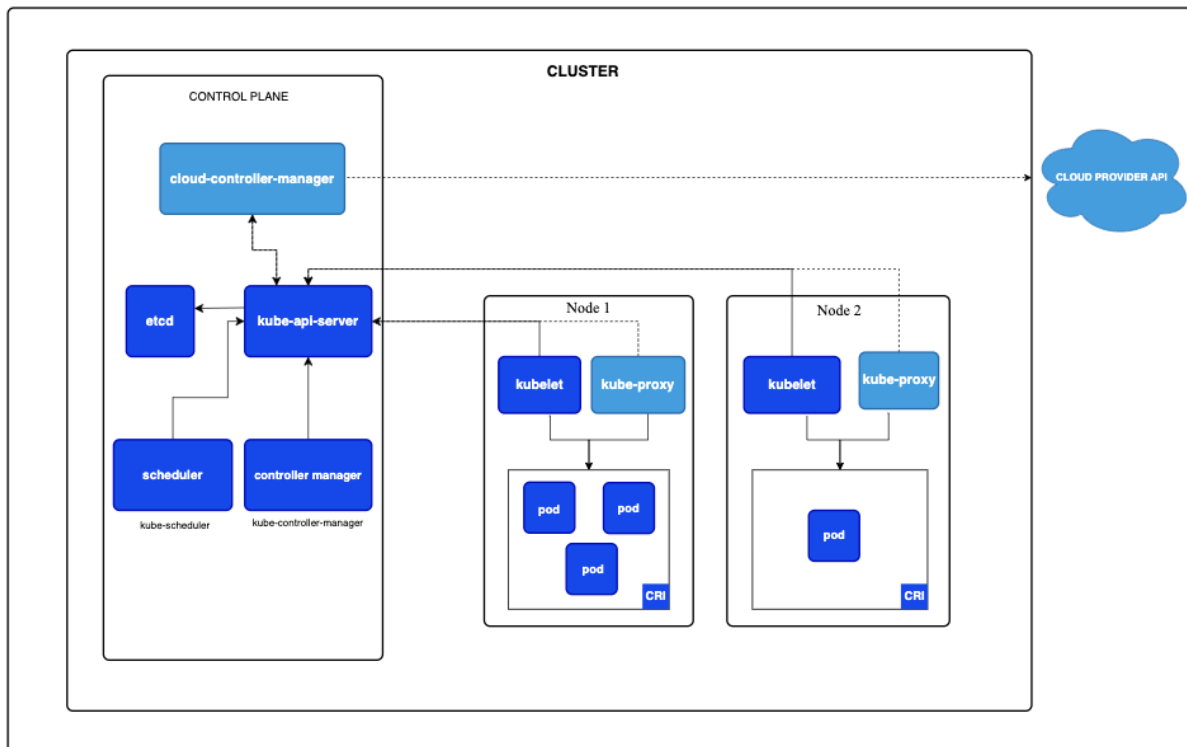


Figure 2-2: Typical Kubernetes Architecture

A Kubernetes cluster typically comprises the Control Plane as well as multiple Worker Nodes. Each of these nodes hold different responsibilities, which are detailed below.

1. Master Node

- (a) The Kubernetes master node, also known as the control plane, is the controller of the whole system. This is where all operational decisions are made, including scheduling, balancing, and responding to events.
- (b) *kube-api-server*: The Kubernetes API Server is the main module within the master node which receives all REST requests for modifications, thereby serving as the frontend to the cluster [4]. It acts as an interface for interaction with the cluster.
- (c) *kube-controller-manager*: The kube-controller-manager runs a distinct number of daemon processes, which help to regulate the shared state of the cluster, as well

as perform routine tasks. When changes in configurations occur, the kcm is responsible for spotting the change and working toward the new desired state [4].

- (d) *Kube-scheduler*: The Kubernetes scheduler helps to schedule the pods on the various nodes based on resource utilization [4]. The kube-scheduler is aware of all the resources available, and is responsible for allocating pods to nodes in an efficient manner to avoid resource exhaustion. Should no resources be available, the kube-scheduler puts pods in a pending state until resources are available again.

2. Worker Nodes

- (a) The Kubernetes worker node, is a worker machine in the Kubernetes cluster. There can be multiple worker nodes in a Kubernetes cluster. Each node is responsible for running pods, and is managed by the master node.
- (b) *kubelet*: The kubelet is the main service on a node, which regularly takes in pod specifications through the kube-api-server. The Kubelet is responsible for ensuring that pods and containers are healthy, and running in the desired state [4]. The Kubelet is also responsible for health checks.
- (c) *kube-proxy*: The kube-proxy is a service that runs on each worker node, that deals with networking. It is responsible for exposing services to the external world, or to the rest of the cluster. It performs request forwarding to the correct pods across various isolated networks in a cluster [4].
- (d) *Container runtime*: The container runtime is responsible for pulling images from a container image registry (i.e. Dockerhub, Amazon AWS ECR), and starting and stopping containers. The most common plugin used for this is Docker [4].

3. Pods

- (a) A pod is the smallest unit that can be scheduled in Kubernetes [4]. Pods help in scalability through replication. In the case of errors or failed health checks, Kubernetes also creates new pods, and destroys unhealthy ones. A single pod can contain multiple containers, and these containers share an IP address and port space, and are able to communicate internally via localhost.

Networking in Kubernetes

Another concept relevant to the development of cloudkubernetes is networking in Kubernetes. The following section will cover two main networking concepts in Kubernetes, which will become relevant to our implementation in Chapter 4.

1. **Ingress**: A Kubernetes ingress object is an API object that manages external access to services in a cluster, typically through HTTP/HTTPS. The Ingress object allows us to map traffic to different backends based on rules defined via the Kubernetes API.
2. **Ingress Controller**: a. In order for an Ingress to work in the cluster, there must be an ingress controller running. Ingress controllers are not started automatically with a cluster, and requires a one to be specified and provisioned for ingress resources to work.

Storage in Kubernetes Lastly, this section will detail how kubernetes handles ephemeral and persistent storage. Similarly, this becomes relevant to our implementation as detailed in Chapter 4.

1. **Volumes:** Volumes in Kubernetes provide a way for containers in a pod to access and share data via the filesystem. This can be used for data sharing between two different containers in the same pod, or even between two different pods. There are two broad categories of volumes. Ephemeral volume types have the lifetime of a pod, and follow the lifecycle of a pod, while persistent volumes exist beyond the lifetime of a pod.
2. **Persistent Volumes (PV):** A persistent volume is a piece of storage in the cluster that has been provisioned by the developer. The data in the PV remains even after pods are destroyed or recreated.
3. **Persistent Volume Claims (PVC):** A Persistent Volume Claim is a request for storage by a user. PVCs consume PV resources, and have defined access modes, such as ReadWriteOnce, ReadOnlyMany, ReadWriteMany, or ReadWriteOncePod.

2.4 Infrastructure and Cloud

This chapter details some of the key cloud infrastructure that is provisioned in order to host speech applications. For a proof of concept, and due to limited access to various clouds, everything in this paper is hosted on the AWS cloud. However, it should be noted that many of these concepts (such as VPC, subnets, load balancers etc.) are similar in other clouds, and cloudkube is extendable to other cloud providers as well. Each subsection will discuss the main uses of each of these components, and briefly detail the corresponding names of these components in AWS, Microsoft Azure Cloud, as well as Google Cloud Provider.

2.4.1 Virtual Private Cloud (VPC)

A virtual private cloud allows the creation of logically isolated sections of the cloud where different resources can be launched in a user-defined virtual network. Developers have full control over the virtual networking environment, including the selection of IP address ranges, creation of subnets, as well as configuration of route tables and network gateways. There will be mentions of some VPC related terminology in the following chapters, all of which are defined here.

- **Subnets:** Subnets allow the division of the VPC into different IP address ranges. They can be public or private.
- **Route Tables:** Route Tables allow the control of routing of traffic within the network. This helps in directing traffic between subnets, or between different VPCs.
- **Internet Gateway (IGW):** The Internet Gateway can be attached to a VPC to allow communication between resources in the VPC and the internet.

-
- **NAT Gateway:** The Network Address Translation (NAT) Gateway enables instances in a private subnet to connect to the internet or other cloud services while preventing the internet from initiating a connection with those instances.
 - **Security Groups / Network Access Control Lists (ACLs):** These are virtual firewalls that control ingress and egress traffic at the instance and subnet levels, providing granular security controls.

Cloud Provider VPCs

- AWS Cloud: AWS Virtual Private Cloud
- GCP: Virtual Private Cloud Network
- Microsoft Azure: Virtual Networks

2.4.2 Subnets

A subnet is a range of IP addresses in a VPC. Subnets allow us to segment the network inside the VPC to group resources and control their access to each other and to the outside world. A subnet can be thought of as a sub-network within a VPC, where resources like EC2 instances can reside.

Key Features of Subnets

1. Public and Private Subnets

- Public Subnets are subnets with a route to an IGW. Instances within this subnet can directly communicate with the internet, provided that they have a public IP address or an Elastic IP.
- Private subnets are subnets which do not have a route to an IGW. Instances in a private subnet cannot communicate with the internet directly. They can only access the internet via a NAT Gateway, or NAT instances in a public subnet.

2. CIDR Blocks (Classless Interdomain Routing)

- Each subnet is associated with a CIDR block that defines its IP address range. CIDR blocks of subnets are always within (longer prefix mask) the CIDR block of the VPC.

3. Availability Zones (AZ)

- Subnets are created within a specific AZ in a region. Resources within a subnet are guaranteed to be located within the same AZ.
- Distribution of subnets across multiple AZs help to increase availability and fault tolerance.

4. NACLs

- Subnets are protected using NACLs, which are stateless firewalls that operate at the subnet level. They control ingress and egress traffic to and from the subnets based on user-defined rules.

2.4.3 Virtual Machines

Kubernetes applications that run in the cloud require compute to be done on virtual machines. Virtual machines are web services offered by cloud providers which allow users to run virtual servers in the cloud. The virtual machines are normally configurable in terms of the choice of processor, storage, networking as well as operating systems.

Cloud Provider Virtual Machines

- AWS Cloud: AWS Elastic Compute Cloud (EC2)
- GCP: Google Compute Engine (GCE)
- Microsoft Azure: Azure Virtual Machine (VM)

Pertinent to this paper, EC2 instances will be provisioned as nodes in the Kubernetes cluster. The Kubernetes pods are then ultimately run on these nodes.

2.4.4 Security Groups

Security groups are virtual firewalls that control inbound and outbound traffic to and from resources. Security groups are stateful, meaning that if you allow an incoming request from a particular IP address and port, the response is automatically allowed without needing a separate outbound rule. Some cloud providers such as Azure also draw a distinction between security groups which operate at the application layer (operating on compute instances and services), as well as the network layer (operating on ingress and egress IP addresses and protocols).

2.4.5 Elastic Network Interfaces (ENI)

An ENI is a logical networking component in a VPC that represents a virtual network card. These ENIs are attached to instances launched in the same AZ. Every VM instance has a primary ENI attached by default, providing the instance with networking capabilities. These ENIs have multiple use cases. Pertinent to this paper, the ENIs are used to facilitate communication between the Kubernetes control plane and worker nodes within the cluster.

2.4.6 Instance Templates

Instance Templates allow users to define the configuration for launch VM instances in a more flexible and reusable way. They allow the specification of instance parameters such as instance type, key pair, security groups and more. These templates can then be used by Auto Scaling Groups, Spot Fleets, and other services to provision VM instances according to the configurations specified.

Cloud Provider Instance Templates

- AWS Cloud: AWS Launch Template

-
- GCP: Instance Templates

Appendix A1 shows an example AWS Launch Template to provision a VPC for use with Kubernetes.

2.4.7 Managed Kubernetes Service

Managed Kubernetes Services are services offered by cloud providers which can be used to run Kubernetes without needing to install, operate, and maintain your own Kubernetes control plane or nodes. The managed Kubernetes service is able to automatically scale control plane instances based on load, detect and replace unhealthy control plane instances, and provide automated version updates and patching for them. These managed service also often offer easy integrability with other cloud services such as file systems for persistent volume storage, and load balancing for load distribution.

Cloud Provider Managed Kubernetes Service

- AWS Cloud: Elastic Kubernetes Service (EKS)
- GCP: Google Kubernetes Engine (GKE)
- Microsoft Azure: Azure Kubernetes Service (AKS)

2.4.8 File Systems

File systems provide a simple, serverless, set-and-forget elastic file system for use with cloud resources or on premise infrastructure. They are built to scale on demand to petabytes without disrupting applications, growing and shrinking automatically as files are added or removed, thereby eliminating the need to provision and manage capacity to accommodate growth.

Cloud Provider File Systems

- AWS Cloud: Elastic File System (EFS)
- GCP: Cloud Filestore
- Microsoft Azure: Fileshare

2.4.9 Scaling Groups

Often times, it is important for compute instances to automatically scale with load. Scaling groups are a collection of compute instances treated as a logical grouping. Its main purpose and objective is to provide for automatic scaling and management. This allows the user to maintain the number of instances in a scaling group. The scaling group starts by launching enough instances to meet its desired capacity. It then maintains this number of instances by conducting periodic health checks on instances in the group. If an instance becomes unhealthy, the scaling group then terminates the instance and launches another instance to replace it.

Cloud Provider Scaling Groups

- AWS Cloud: Auto Scaling Groups (ASG)
- GCP: Managed Instance Groups (MIG)
- Microsoft Azure: Azure Virtual Machine Scale Sets (VMSS)

2.4.10 minikube

minikube is a way for one to quickly set up a local Kubernetes cluster. It is compatible with macOS, Linux and Windows. The main goal of minikube is to help application developers do quick testing of their Kubernetes deployments locally, as well as a tool used for learning by new Kubernetes users [11]. By default, minikube creates a one-node cluster locally, and supports addons such as ingress controllers. As we will see in the following chapter, cloudkubernetes functionality closely mirrors that of minikube, supporting the objective of cloudkubernetes which is to provide a quick and easy testing environment for Kubernetes developers in the cloud.

Planning and Design

3.1 Traditional Approaches

This section explores existing solutions like AWS ECS...

Implementation

Details on CloudKube's architecture and workflow...

Results and Analysis

Analysis of deployment times, cost savings, and usability...

Conclusion and Future Work

CloudKube simplifies Kubernetes deployment...

Appendix A: AWS CloudFormation YAML Template for EKS IPv4

The following is the YAML template used for provisioning the infrastructure for an EKS compliant VPC (IPv4):

```
1  ---
2  AWSTemplateFormatVersion: '2010-09-09'
3  Description: 'Amazon EKS Sample VPC - Private and Public subnets'
4
5  Parameters:
6
7    VpcBlock:
8      Type: String
9      Default: 192.168.0.0/16
10     Description: The CIDR range for the VPC. This should be a valid private (RFC
11       1918) CIDR range.
12
13     PublicSubnet01Block:
14       Type: String
15       Default: 192.168.0.0/18
16       Description: CidrBlock for public subnet 01 within the VPC
17
18     PublicSubnet02Block:
19       Type: String
20       Default: 192.168.64.0/18
21       Description: CidrBlock for public subnet 02 within the VPC
22
23     PrivateSubnet01Block:
24       Type: String
25       Default: 192.168.128.0/18
26       Description: CidrBlock for private subnet 01 within the VPC
27
28     PrivateSubnet02Block:
29       Type: String
30       Default: 192.168.192.0/18
31       Description: CidrBlock for private subnet 02 within the VPC
32
33  Metadata:
34    AWS::CloudFormation::Interface:
35      ParameterGroups:
36        -
37          Label:
38            default: "Worker Network Configuration"
39          Parameters:
40            - VpcBlock
41            - PublicSubnet01Block
42            - PublicSubnet02Block
43            - PrivateSubnet01Block
44            - PrivateSubnet02Block
45
46  Resources:
47    VPC:
48      Type: AWS::EC2::VPC
49      Properties:
```

```

49     CidrBlock: !Ref VpcBlock
50     EnableDnsSupport: true
51     EnableDnsHostnames: true
52     Tags:
53     - Key: Name
54       Value: !Sub '${AWS::StackName}-VPC'
55
56   InternetGateway:
57     Type: "AWS::EC2::InternetGateway"
58
59   VPCGatewayAttachment:
60     Type: "AWS::EC2::VPCGatewayAttachment"
61     Properties:
62       InternetGatewayId: !Ref InternetGateway
63       VpcId: !Ref VPC
64
65   PublicRouteTable:
66     Type: AWS::EC2::RouteTable
67     Properties:
68       VpcId: !Ref VPC
69       Tags:
70       - Key: Name
71         Value: Public Subnets
72       - Key: Network
73         Value: Public
74
75   PrivateRouteTable01:
76     Type: AWS::EC2::RouteTable
77     Properties:
78       VpcId: !Ref VPC
79       Tags:
80       - Key: Name
81         Value: Private Subnet AZ1
82       - Key: Network
83         Value: Private01
84
85   PrivateRouteTable02:
86     Type: AWS::EC2::RouteTable
87     Properties:
88       VpcId: !Ref VPC
89       Tags:
90       - Key: Name
91         Value: Private Subnet AZ2
92       - Key: Network
93         Value: Private02
94
95   PublicRoute:
96     DependsOn: VPCGatewayAttachment
97     Type: AWS::EC2::Route
98     Properties:
99       RouteTableId: !Ref PublicRouteTable
100       DestinationCidrBlock: 0.0.0.0/0
101       GatewayId: !Ref InternetGateway
102
103   PrivateRoute01:
104     DependsOn:
105     - VPCGatewayAttachment
106     - NatGateway01
107     Type: AWS::EC2::Route
108     Properties:
109       RouteTableId: !Ref PrivateRouteTable01
110       DestinationCidrBlock: 0.0.0.0/0
111       NatGatewayId: !Ref NatGateway01

```

```

112
113 PrivateRoute02:
114   DependsOn:
115     - VPCGatewayAttachment
116     - NatGateway02
117   Type: AWS::EC2::Route
118   Properties:
119     RouteTableId: !Ref PrivateRouteTable02
120     DestinationCidrBlock: 0.0.0.0/0
121     NatGatewayId: !Ref NatGateway02
122
123 NatGateway01:
124   DependsOn:
125     - NatGatewayEIP1
126     - PublicSubnet01
127     - VPCGatewayAttachment
128   Type: AWS::EC2::NatGateway
129   Properties:
130     AllocationId: !GetAtt 'NatGatewayEIP1.AllocationId'
131     SubnetId: !Ref PublicSubnet01
132     Tags:
133       - Key: Name
134         Value: !Sub '${AWS::StackName}-NatGatewayAZ1'
135
136 NatGateway02:
137   DependsOn:
138     - NatGatewayEIP2
139     - PublicSubnet02
140     - VPCGatewayAttachment
141   Type: AWS::EC2::NatGateway
142   Properties:
143     AllocationId: !GetAtt 'NatGatewayEIP2.AllocationId'
144     SubnetId: !Ref PublicSubnet02
145     Tags:
146       - Key: Name
147         Value: !Sub '${AWS::StackName}-NatGatewayAZ2'
148
149 NatGatewayEIP1:
150   DependsOn:
151     - VPCGatewayAttachment
152   Type: 'AWS::EC2::EIP'
153   Properties:
154     Domain: vpc
155
156 NatGatewayEIP2:
157   DependsOn:
158     - VPCGatewayAttachment
159   Type: 'AWS::EC2::EIP'
160   Properties:
161     Domain: vpc
162
163 PublicSubnet01:
164   Type: AWS::EC2::Subnet
165   Metadata:
166     Comment: Subnet 01
167   Properties:
168     MapPublicIpOnLaunch: true
169     AvailabilityZone:
170       Fn::Select:
171         - '0'
172       - Fn::GetAZs:
173         Ref: AWS::Region
174     CidrBlock:

```

```

175         Ref: PublicSubnet01Block
176     VpcId:
177         Ref: VPC
178     Tags:
179     - Key: Name
180       Value: !Sub "${AWS::StackName}-PublicSubnet01"
181     - Key: kubernetes.io/role/elb
182       Value: 1
183
184 PublicSubnet02:
185     Type: AWS::EC2::Subnet
186     Metadata:
187         Comment: Subnet 02
188     Properties:
189         MapPublicIpOnLaunch: true
190         AvailabilityZone:
191             Fn::Select:
192             - '1'
193             - Fn::GetAZs:
194                 Ref: AWS::Region
195         CidrBlock:
196             Ref: PublicSubnet02Block
197         VpcId:
198             Ref: VPC
199         Tags:
200         - Key: Name
201           Value: !Sub "${AWS::StackName}-PublicSubnet02"
202         - Key: kubernetes.io/role/elb
203           Value: 1
204
205 PrivateSubnet01:
206     Type: AWS::EC2::Subnet
207     Metadata:
208         Comment: Subnet 03
209     Properties:
210         AvailabilityZone:
211             Fn::Select:
212             - '0'
213             - Fn::GetAZs:
214                 Ref: AWS::Region
215         CidrBlock:
216             Ref: PrivateSubnet01Block
217         VpcId:
218             Ref: VPC
219         Tags:
220         - Key: Name
221           Value: !Sub "${AWS::StackName}-PrivateSubnet01"
222         - Key: kubernetes.io/role/internal-elb
223           Value: 1
224
225 PrivateSubnet02:
226     Type: AWS::EC2::Subnet
227     Metadata:
228         Comment: Private Subnet 02
229     Properties:
230         AvailabilityZone:
231             Fn::Select:
232             - '1'
233             - Fn::GetAZs:
234                 Ref: AWS::Region
235         CidrBlock:
236             Ref: PrivateSubnet02Block
237         VpcId:

```

```

238         Ref: VPC
239     Tags:
240     - Key: Name
241       Value: !Sub "${AWS::StackName}-PrivateSubnet02"
242     - Key: kubernetes.io/role/internal-elb
243       Value: 1
244
245     PublicSubnet01RouteTableAssociation:
246       Type: AWS::EC2::SubnetRouteTableAssociation
247       Properties:
248         SubnetId: !Ref PublicSubnet01
249         RouteTableId: !Ref PublicRouteTable
250
251     PublicSubnet02RouteTableAssociation:
252       Type: AWS::EC2::SubnetRouteTableAssociation
253       Properties:
254         SubnetId: !Ref PublicSubnet02
255         RouteTableId: !Ref PublicRouteTable
256
257     PrivateSubnet01RouteTableAssociation:
258       Type: AWS::EC2::SubnetRouteTableAssociation
259       Properties:
260         SubnetId: !Ref PrivateSubnet01
261         RouteTableId: !Ref PrivateRouteTable01
262
263     PrivateSubnet02RouteTableAssociation:
264       Type: AWS::EC2::SubnetRouteTableAssociation
265       Properties:
266         SubnetId: !Ref PrivateSubnet02
267         RouteTableId: !Ref PrivateRouteTable02
268
269     ControlPlaneSecurityGroup:
270       Type: AWS::EC2::SecurityGroup
271       Properties:
272         GroupDescription: Cluster communication with worker nodes
273         VpcId: !Ref VPC
274
275     Outputs:
276
277     SubnetIds:
278       Description: Subnets IDs in the VPC
279       Value: !Join [ ",", [ !Ref PublicSubnet01, !Ref PublicSubnet02, !Ref
280         PrivateSubnet01, !Ref PrivateSubnet02 ] ]
281
282     SecurityGroups:
283       Description: Security group for the cluster control plane communication with
284         worker nodes
285       Value: !Join [ ",", [ !Ref ControlPlaneSecurityGroup ] ]
286
287     VpcId:
288       Description: The VPC Id
289       Value: !Ref VPC

```

Listing A.1: AWS CloudFormation YAML Template for EKS IPv4